

Laboratorio de Algoritmos

Funciones, parámetros, vectores y matrices

Ing. Jorge Luis García Chacón

Tabla de contenidos

1	Propósito del laboratorio	1
1.1	Reglas	2
2	Ejercicio 1. Campeonato de Videojuegos	2
2.1	Clasificación	2
2.2	Requerimientos	3
2.3	Funciones	3
2.4	Antes de programar	3
3	Ejercicio 2. Gran Premio de Motociclismo	4
3.1	Ejemplo conceptual	4
3.2	Promedio y clasificación	4
3.3	Requerimientos	5
3.4	Funciones	5
3.5	Antes de programar	5
4	Recomendaciones generales	6

1. Propósito del laboratorio

El propósito de este laboratorio es integrar los conceptos estudiados durante el curso mediante la resolución de problemas completos.

En ambos ejercicios deberá analizar el problema antes de iniciar la programación. La solución deberá utilizar funciones con parámetros, estructuras de control y estructuras para almacenar varios datos.

1.1. Reglas

- Carné terminado en número impar (1, 3, 5, 7, 9): Ejercicio 1 – Campeonato de Videojuegos.
- Carné terminado en número par (0, 2, 4, 6, 8): Ejercicio 2 – Gran Premio de Motociclismo.

! Importante

No se proporciona código inicial.

Antes de programar, analice el problema y determine qué variables, estructuras de control, ciclos, funciones y estructuras de almacenamiento necesita.

El programa deberá funcionar correctamente para todos los casos planteados.

2. Ejercicio 1. Campeonato de Videojuegos

Una empresa organiza un campeonato de videojuegos en el que participan **5 jugadores**.

De cada jugador se deberá registrar:

- Nombre.
- Partidas ganadas.
- Partidas empatadas.
- Partidas perdidas.

La información de los participantes deberá almacenarse utilizando **vectores o arreglos**.

Cada victoria otorga **5 puntos**, cada empate **2 puntos** y cada derrota **0 puntos**.

La puntuación total se calculará de acuerdo con:

$$[\text{Puntos} = (\text{Victorias} \times 5) + (\text{Empates} \times 2)]$$

2.1. Clasificación

Según el puntaje total obtenido, cada jugador deberá recibir una clasificación:

Puntaje	Clasificación
Menos de 15 puntos	Bronce
De 15 a 29 puntos	Plata
De 30 a 44 puntos	Oro
45 puntos o más	Leyenda

2.2. Requerimientos

El programa deberá:

1. Registrar la información de los **5 jugadores**.
2. Almacenar los datos utilizando **vectores o arreglos**.
3. Calcular el puntaje total de cada jugador.
4. Determinar la clasificación correspondiente.
5. Mostrar los resultados de todos los participantes.
6. Determinar y mostrar el jugador que obtuvo el **mayor puntaje**.
7. Validar que las cantidades de partidas ganadas, empatadas y perdidas no sean negativas.
8. Utilizar ciclos para registrar y recorrer los datos almacenados.

2.3. Funciones

La solución deberá utilizar **como mínimo dos funciones con parámetros**:

- Una función encargada de calcular y devolver el puntaje de un jugador.
- Una función encargada de determinar y devolver la clasificación correspondiente.

El estudiante puede utilizar funciones adicionales si considera que mejoran la organización del programa.

2.4. Antes de programar

Realice:

1. Entradas.
2. Procesos.
3. Salidas.
4. Pseudocódigo (opcional).
5. Diagrama de flujo (opcional).

i Pregunta para analizar

Si tenemos cinco jugadores, ¿qué ventaja obtenemos al almacenar la información en arreglos en lugar de crear variables independientes para cada jugador?

3. Ejercicio 2. Gran Premio de Motociclismo

Se organiza un campeonato de motociclismo en el que participan **5 pilotos** y se disputan **3 carreras**.

El programa deberá solicitar el nombre de cada piloto y registrar el tiempo, expresado en minutos, obtenido por cada piloto en cada una de las tres carreras.

Los tiempos deberán almacenarse utilizando una **matriz**, donde:

- Cada **fila** represente un piloto.
- Cada **columna** represente una carrera.

3.1. Ejemplo conceptual

Piloto	Carrera 1	Carrera 2	Carrera 3
Piloto 1	Tiempo	Tiempo	Tiempo
Piloto 2	Tiempo	Tiempo	Tiempo
Piloto 3	Tiempo	Tiempo	Tiempo
Piloto 4	Tiempo	Tiempo	Tiempo
Piloto 5	Tiempo	Tiempo	Tiempo

El estudiante deberá determinar cómo declarar y recorrer esta estructura dentro del programa.

3.2. Promedio y clasificación

Después de registrar las tres carreras, deberá calcularse el **tiempo promedio de cada piloto**.

Según su promedio, cada piloto recibirá una categoría:

Tiempo promedio	Categoría
Hasta 40 minutos	Élite
Más de 40 y hasta 50 minutos	Profesional
Más de 50 y hasta 60 minutos	Intermedio
Más de 60 minutos	Fuera de clasificación

En una carrera gana quien registra el menor tiempo. Por lo tanto, después de calcular todos los promedios, el programa deberá determinar qué piloto obtuvo el **menor tiempo promedio**.

3.3. Requerimientos

El programa deberá:

1. Registrar los nombres de los **5 pilotos**.
2. Registrar los tiempos de las **3 carreras** de cada piloto.
3. Almacenar los tiempos utilizando una **matriz**.
4. Utilizar ciclos anidados para registrar y recorrer la matriz.
5. Calcular el tiempo promedio de cada piloto.
6. Determinar la categoría correspondiente.
7. Mostrar los resultados de todos los pilotos.
8. Determinar y mostrar el piloto con el **mejor promedio**.
9. No permitir tiempos iguales o menores que cero.

3.4. Funciones

La solución deberá utilizar **como mínimo dos funciones con parámetros**:

- Una función encargada de calcular y devolver el promedio de un piloto.
- Una función encargada de determinar y devolver la categoría correspondiente.

El estudiante puede agregar otras funciones si considera que ayudan a organizar mejor el programa.

3.5. Antes de programar

Realice:

1. Entradas.
2. Procesos.
3. Salidas.
4. Pseudocódigo (opcional).
5. Diagrama de flujo (opcional).

i Pregunta para analizar

¿Por qué una matriz resulta apropiada para representar los resultados de varios pilotos en varias carreras?

4. Recomendaciones generales

- Analice antes de programar.
- Utilice nombres de variables comprensibles.
- Evite repetir código innecesariamente.
- Cada función debe tener una responsabilidad clara.
- Utilice parámetros para enviar a las funciones los datos que necesitan.
- Pruebe el programa con diferentes valores.
- Verifique especialmente los casos límite de cada clasificación.

Para el laboratorio

El objetivo no es solamente obtener un programa que funcione. La solución debe demostrar que el problema fue **analizado, organizado y dividido en responsabilidades** antes de escribir el código.